

Paradigmi di Programmazione

Diego Stefanini — prof.ssa Chiara Bodei, a.a. 2026-27

Indice

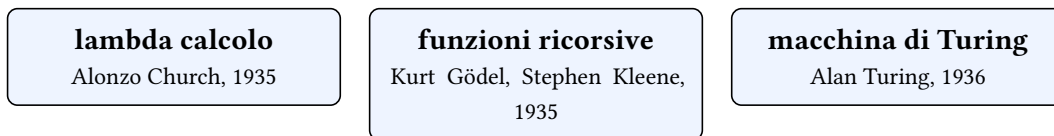
1 Il lambda calcolo	3
1.1 Calcolabilità e paradigmi	3
1.2 Sintassi dei λ -termini	4
1.3 Esempi di calcolo	6
1.4 Variabili libere e legate	7

1 Il lambda calcolo

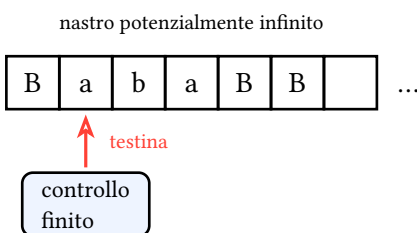
1.1 Calcolabilità e paradigmi

Hilbert, Entscheidungsproblem (problema della decisione): esiste una procedura **del tutto meccanica** che, data una qualunque formula della logica del primo ordine, dice se è un teorema?

Per rispondere bisogna prima dire cos'è una «procedura meccanica», cioè un **algoritmo**. Tre risposte diverse, quasi insieme:



Com'è fatta la **macchina di Turing**:

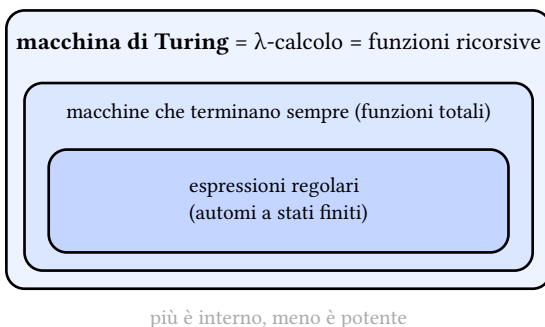


- Macchina **ideale**, non fisica: legge e scrive simboli su un nastro, secondo regole fissate.
- Esempio di regola: *legge a, riscrive a e si sposta a destra.*
- È il modello astratto di una macchina che esegue algoritmi.

Macchina di Turing Universale (UTM): una macchina di Turing che, data la **descrizione** di un'altra macchina e i suoi dati, la simula. È il modello teorico del computer: il programma è un dato.

Con la UTM Turing risponde **no** all'Entscheidungsproblem: esistono problemi che nessuna macchina di calcolo può risolvere.

I tre formalismi calcolano esattamente le stesse funzioni. Da qui la **tesi di Church-Turing**:

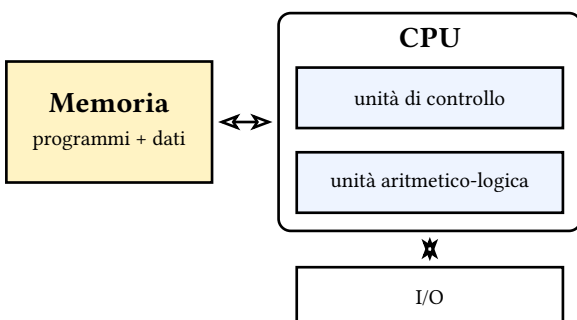


Se una funzione è calcolabile con un qualunque formalismo, allora esiste una macchina di Turing che la calcola.

È **indimostrabile** (i formalismi possibili sono infiniti) ma universalmente accettata: non si conosce nessun modello più potente.

Un linguaggio è **Turing completo** se calcola tutto ciò che calcola una macchina di Turing. Si dimostra scrivendo nel linguaggio un programma che **simula** una macchina di Turing.

Dai due modelli, Turing e Church, nascono due famiglie di linguaggi. Da Turing vengono i linguaggi **imperativi**, passando per la macchina di von Neumann:



Architettura di von Neumann, ispirata alla macchina di Turing ma concreta:

- **memoria** con programmi e dati;
- **CPU** che preleva un'istruzione alla volta, la interpreta e la esegue.

La generalità viene dal **programma memorizzato**.

Un linguaggio «alla von Neumann» riproduce ad alto livello questa struttura:

Nel linguaggio		Nella macchina
variabili	↔	celle di memoria (il nastro)
istruzioni di controllo (if, cicli)	↔	test & jump
assegnamento	↔	modifica dello stato (fetch & store)

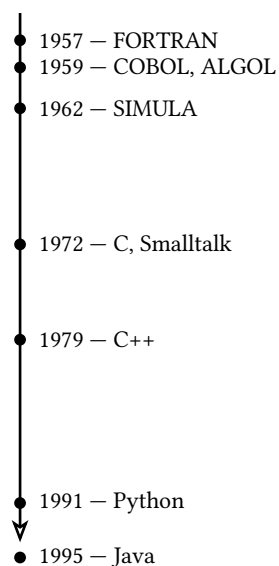
Backus: l'assegnamento divide la programmazione in due mondi.

espressioni spazio matematico ordinato, con proprietà algebriche utili. Lì avviene la maggior parte del calcolo	istruzioni spazio disordinato, con poche proprietà utili. La programmazione strutturata prova a metterci un po' d'ordine
--	---

Da Church vengono invece i linguaggi **funzionali**.

Programmazione funzionale: il programma è una serie di **valutazioni di funzioni matematiche**. Punto di forza: niente **effetti collaterali**, quindi è più facile verificare che il programma sia corretto e ottimizzarlo. Il λ -calcolo (Church, 1935) è il primo linguaggio funzionale.

linguaggi «di Turing»



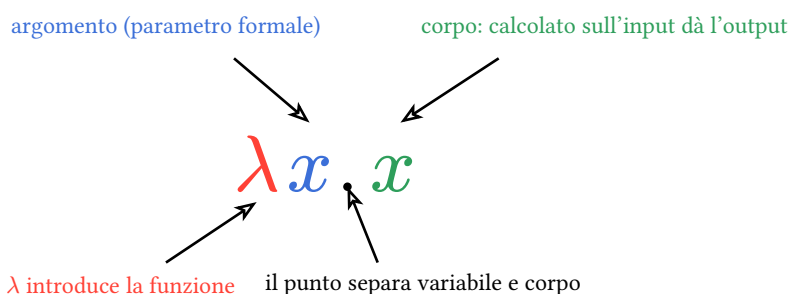
linguaggi «di Church»



Le due linee di discendenza

1.2 Sintassi dei λ -termini

Il λ -calcolo ha solo due operazioni. La prima è l'**astrazione**, cioè definire una funzione:



Legge di corrispondenza: $\forall x. f(x) = x$. È la **funzione identità**.

La seconda è l'**applicazione**. Applicare una funzione = darle un **parametro attuale** al posto del parametro formale. Immagina la funzione come una scatola:

$z \rightarrow \boxed{\lambda x.x} \rightarrow z$ **identità**: restituisce l'input così com'è

$z \rightarrow \boxed{\lambda x.y} \rightarrow y$ **costante**: restituisce sempre y ($\forall x.f(x) = y$)

$z, w \rightarrow \boxed{\lambda x.\lambda y.x} \rightarrow z$ **selezione**: prende due argomenti e restituisce il primo

La selezione passo per passo: $((\lambda x.\lambda y.x)z)w \rightarrow (\lambda y.z)w \rightarrow z$.

$\lambda x.\lambda y.x$ ha legge di corrispondenza $\forall x.f(x) = g$ dove $g(y) = x$: una funzione che restituisce una funzione. È un modo alternativo di scrivere $f(x, y) = x$ con una funzione di un solo argomento.

Queste due operazioni, più le variabili, sono tutto il linguaggio. Un programma è un'espressione (**λ -espressione**). Ci sono solo tre modi di costruirla:

$e ::= x$ variabile
 | $\lambda x.e$ astrazione funzionale (dichiarazione di funzione)
 | $e e$ applicazione (chiamata di funzione)

Niente altro! La sintassi è finita.

$\lambda x.e$ è una **funzione anonima**: non ha nome, e x è la dichiarazione del suo parametro. Lo stesso concetto nei linguaggi:

Linguaggio	$x \mapsto x + 1$
JavaScript	<code>function(a){ return a + 1; }</code> oppure <code>a => a+1</code>
OCaml	<code>fun x -> x+1</code>
Java (si chiamano lambda)	<code>(int x) -> x + 1</code>

In $e_1 e_2$ la funzione e_1 è applicata all'argomento e_2 , come una chiamata in JavaScript: e_1 definisce la funzione, e_2 il parametro attuale. La definizione può stare **direttamente dentro** la chiamata:

$\underbrace{(\lambda x.(\lambda y.xy))}_{e_1} \underbrace{(\lambda z.z)}_{e_2}.$

Per non scrivere troppe parentesi valgono due convenzioni (importanti):

1. Lo scope di λ va il più a destra possibile

$\lambda x.\lambda y.xy$ è $\lambda x.(\lambda y.(xy))$

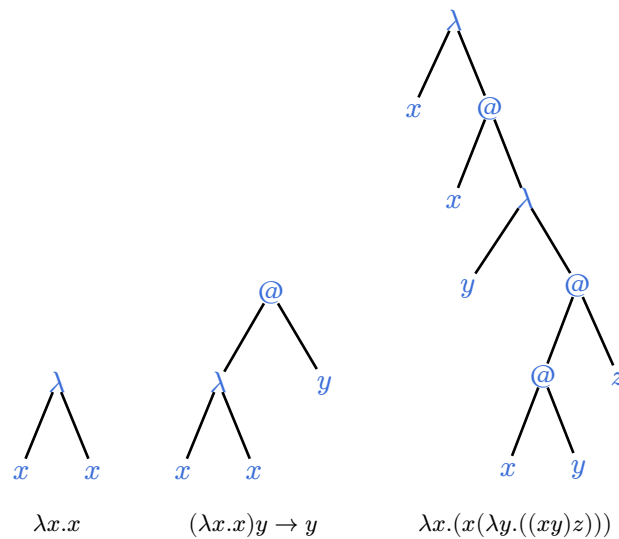
2. L'applicazione associa a sinistra

$e_1 e_2 e_3$ è $(e_1 e_2) e_3$

Esercizio: quali parentesi sono sottintese in $\lambda x.x \lambda y.x y z$?

1. lo scope di λ va il più a destra possibile: $\lambda x.(x \lambda y.(x y z))$
2. l'applicazione associa a sinistra: $\lambda x.(x (\lambda y.((x y) z)))$

Ogni termine si può disegnare come un **albero**, che segue l'annidamento completo delle parentesi. Serve a vedere quali passi di valutazione si possono fare. Nodo λ : figlio sinistro il parametro, destro il corpo. Nodo $@$ (applicazione): figli funzione e argomento.



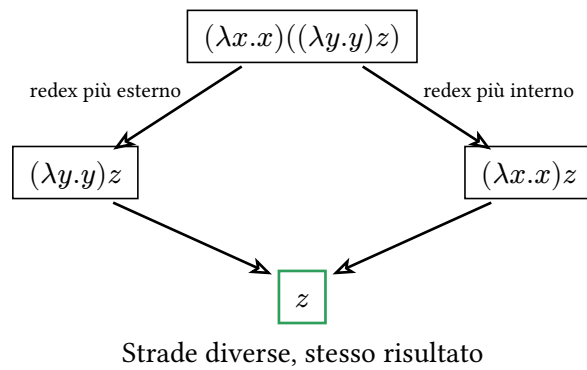
1.3 Esempi di calcolo

Espressione	Cosa fa
$\lambda x.x$	identità
$\lambda y.(\lambda x.x)$	scarta l'argomento y e restituisce l'identità: è una funzione costante
$\lambda f.f(\lambda x.x)$	data una funzione f , la invoca sull'identità

Applicazione	Passi
$(\lambda x.x)y$	$\rightarrow y$
$(\lambda x.((\lambda y.y)x))z$	$\rightarrow (\lambda y.y)z \rightarrow z$
$(\lambda f.fz)(\lambda x.x)$	$\rightarrow (\lambda x.x)z \rightarrow z$ prende una funzione e la applica a z
$(\lambda x.(xx))(\lambda y.y)$	$\rightarrow (\lambda y.y)(\lambda y.y) \rightarrow \lambda y.y$ due termini uguali con ruoli diversi: funzione e argomento
$((\lambda x.\lambda y.x + y)3)5$	$\rightarrow (\lambda y.3 + y)5 \rightarrow 3 + 5 \rightarrow 8$ (supponendo di avere il $+$)
$((\lambda x.\lambda y.xy)(\lambda x.x))z$	$\rightarrow (\lambda y.(\lambda x.x)y)z \rightarrow (\lambda x.x)z \rightarrow z$ forma $e_1 e_2 e_3$
$(\lambda x.\lambda y.xy)zk$	$\rightarrow (\lambda y.zy)k \rightarrow zk$

$(\lambda y.3 + y)$ è una funzione a sé: somma 3 a quello che le passi. Applicando una funzione di due argomenti a uno solo ottieni una funzione che aspetta l'altro.

Spesso si può scegliere da che parte cominciare. Un **redex** è un pezzo della forma $(\lambda x.e)e'$, pronto per essere applicato. In $(\lambda x.x)((\lambda y.y)z)$ ce ne sono due:



1.4 Variabili libere e legate

λ è un **operatore di binding**: in $\lambda x.e$ lega la x dentro e (il suo **scope**), come $\forall x$ in $\forall x.P$ nella logica. È l'**unico** modo di associare valori a variabili nel λ -calcolo.

- **Legata**: introdotta da un λ che la contiene. **verde**, la freccia punta al suo λ .
- **Libera**: nessun λ la dichiara. **rossa**.

Espressione	Chi è legato
$\lambda \mathbf{x} . \mathbf{x}$	x legata
$\lambda \mathbf{x} . \lambda \mathbf{y} . (\mathbf{x} \mathbf{y} \mathbf{z})$	x e y legate, z libera
$(\lambda \mathbf{f} . \mathbf{f} \mathbf{x}) \mathbf{y}$	f legata, x e y libere
$(\lambda \mathbf{f} . \mathbf{f} \mathbf{x}) \mathbf{f}$	la prima f è legata; x e la seconda f sono libere : sta fuori dalle parentesi, fuori dallo scope di λf

Una stessa variabile può voler dire cose diverse in punti diversi:

$$\lambda \mathbf{x} . \mathbf{x} (\lambda \mathbf{x} . \mathbf{x}) \mathbf{x}$$

che con le parentesi esplicite è $\lambda x.(x(\lambda x.x)x)$

La x nel λx interno è legata a **quel** λ , non a quello esterno. Tre x , due significati: da qui nasce il conflitto di nomi.

Ecco il problema che questo crea. Applicando senza fare attenzione ai nomi:

	Espressione	Risultato
ingenuo	$(\lambda x.\lambda y.xy)y$	$\lambda y.yy$ ✗ sbagliato
rinomino y in z	$(\lambda x.\lambda z.xz)y$	$\lambda z.yz$ ✓

Le due espressioni di partenza sono **la stessa funzione** (ho solo cambiato il nome del parametro), ma i risultati si comportano in modo diverso. Nel primo caso la y libera che passo come argomento finisce **catturata** dal λy interno: è un **conflitto di nomi**.

x in $\lambda x.e$ è un segnaposto: si può rinominare con una variabile **fresca**, cioè un nome che non compare da nessuna parte nelle espressioni che stiamo trattando. È così che si risolve il conflitto di nomi, come nella riga «rinomino» della tabella.